



CSE471 : System Analysis and Design
Lab Assignment on API Postman Collection

Project Title : Smart Local Service Intelligence Platform

Group Number : 07

Backend Framework : MERN

Submitted by : Shees Mohammad Hossain Saud

Student ID : 20101234

Section: 12

Date :03/12/2026

APIs of Feature - 01

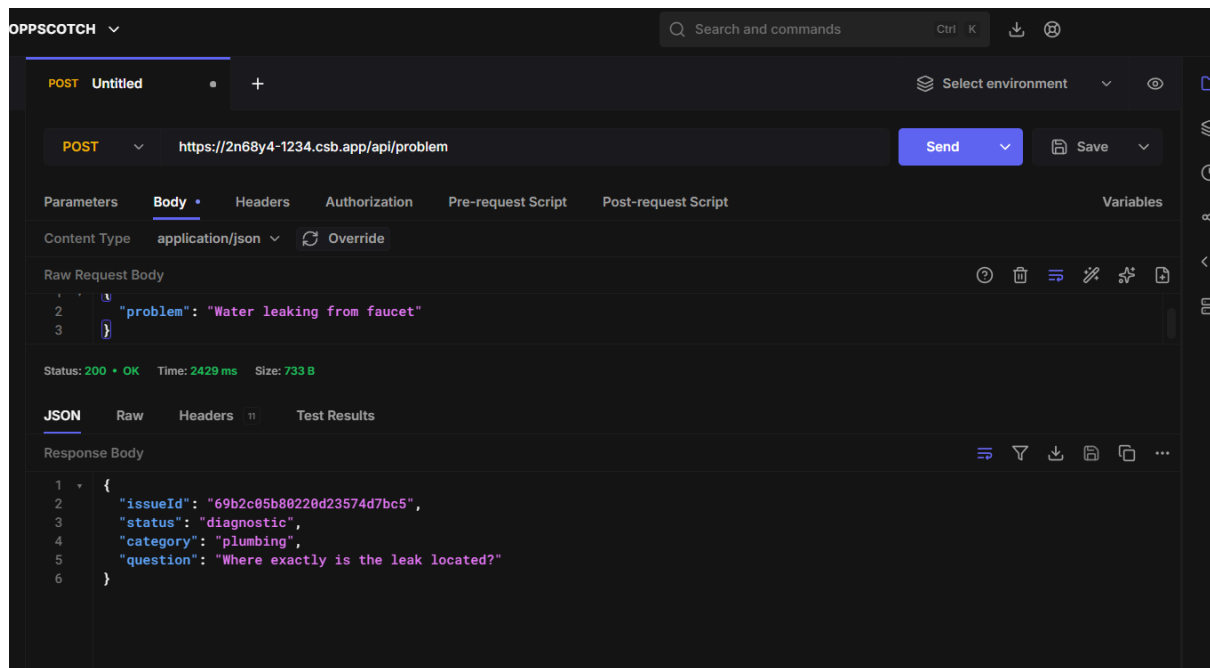
Text Based Issue Submission

```
API 1: SUBMIT PROBLEM
*/
app.post("/api/problem", async (req, res) => {
  try {
    const { problem } = req.body;
    const analysis = await analyzeProblem(problem);

    const issue = new Issue({
      problem,
      category: analysis.category,
      confidence: analysis.confidence,
      status: analysis.status,
      questions: [analysis.question],
    });

    await issue.save();

    res.json({
      issueId: issue._id,
      status: analysis.status,
      category: analysis.category,
      question: analysis.question,
    });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```



- Endpoint URL: <https://2n68y4-1234.csb.app/api/issues>
- HTTP Method: post
- Headers: None
- Body: {“problem” : “water is leaking from faucet”}

APIs of Feature - 02

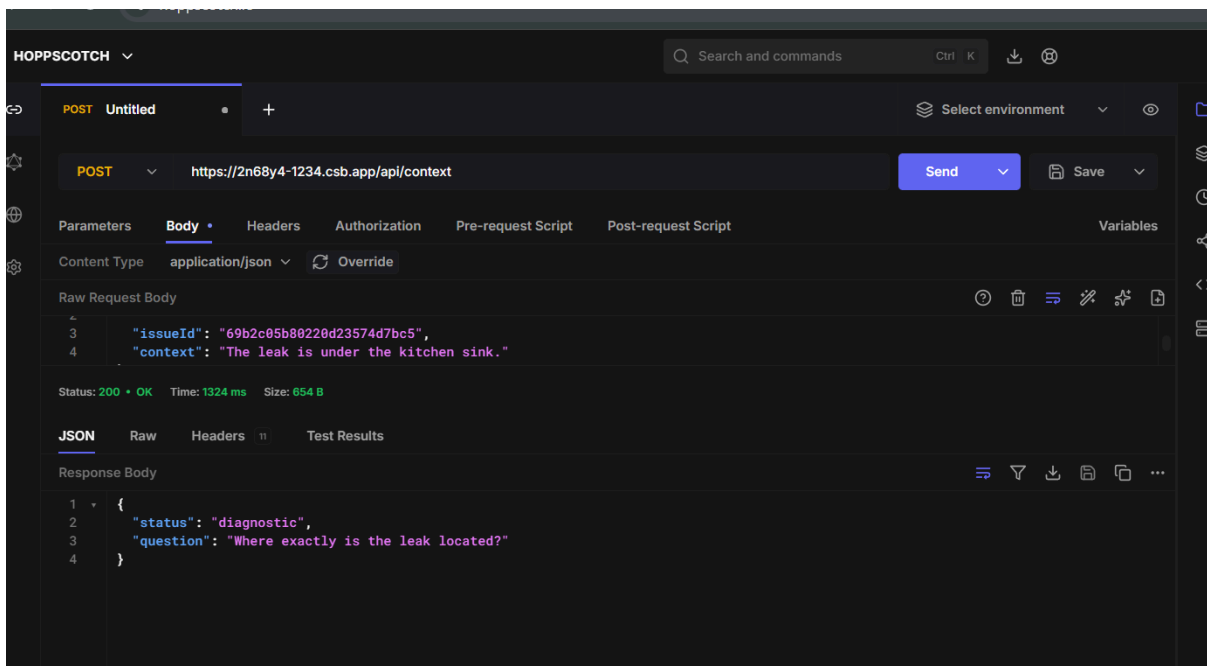
Automatic Issue Categorization

```
API 2: ADD CONTEXT
*/
app.post("/api/context", async (req, res) => {
  try {
    const { issueId, context } = req.body;
    const issue = await Issue.findById(issueId);
    issue.context = context;

    const analysis = await analyzeProblem(issue.problem, context);
    issue.questions.push(analysis.question);

    await issue.save();

    res.json({
      status: analysis.status,
      question: analysis.question,
    });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```



- Endpoint URL: <https://2n68y4-1234.csb.app/api/context>
- HTTP Method: post

- Headers: None
- Body: {


```

        "issueId": "69b2bba480220d23574d7bbb",
        "context": "The leak is under the kitchen sink."
      
```

APIs of Feature - 03

Risk Level Indicator/ Worker Suggestion /DIY

```

app.post("/api/answer", async (req, res) => {
  try {
    const { issueId, answer } = req.body;
    const issue = await Issue.findById(issueId);

    issue.answers.push(answer);
    issue.confidence = Math.min(100, (issue.confidence || 50) + 20);

    const next = await nextQuestion(
      issue.problem,
      issue.category,
      issue.answers
    );

    if (next.askMore && issue.confidence < 85) {
      issue.questions.push(next.question);
      await issue.save();
      return res.json({
        status: "more_questions",
        question: next.question,
      });
    }

    const risk = await evaluateRisk(
      issue.problem,
      issue.category,
      issue.answers
    );
  }
});

```

```

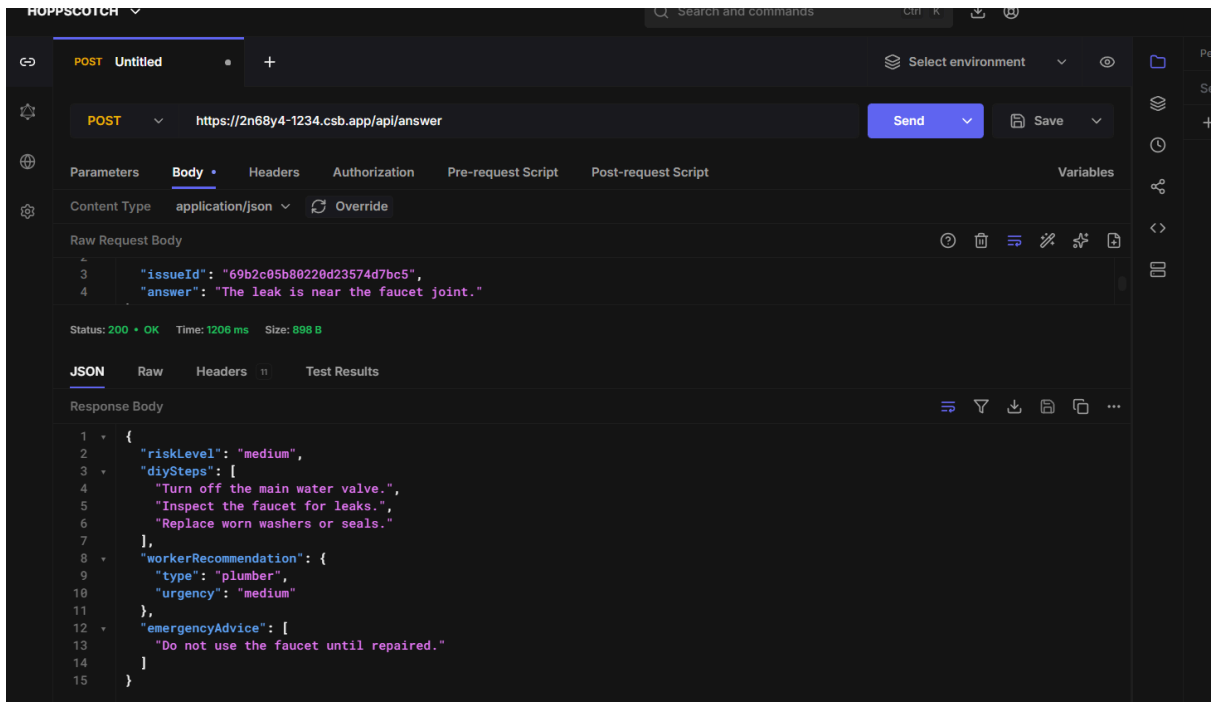
try {
  const risk = await evaluateRisk(
    issue.problem,
    issue.answers
  );
  issue.riskLevel = risk.riskLevel;

  const solution = await generateSolution(
    issue.problem,
    issue.category,
    risk.riskLevel
  );
  issue.diySteps = solution.diySteps;
  issue.workerRecommendation = solution.workerRecommendation;
  issue.emergencyAdvice = solution.emergencyAdvice;

  await issue.save();

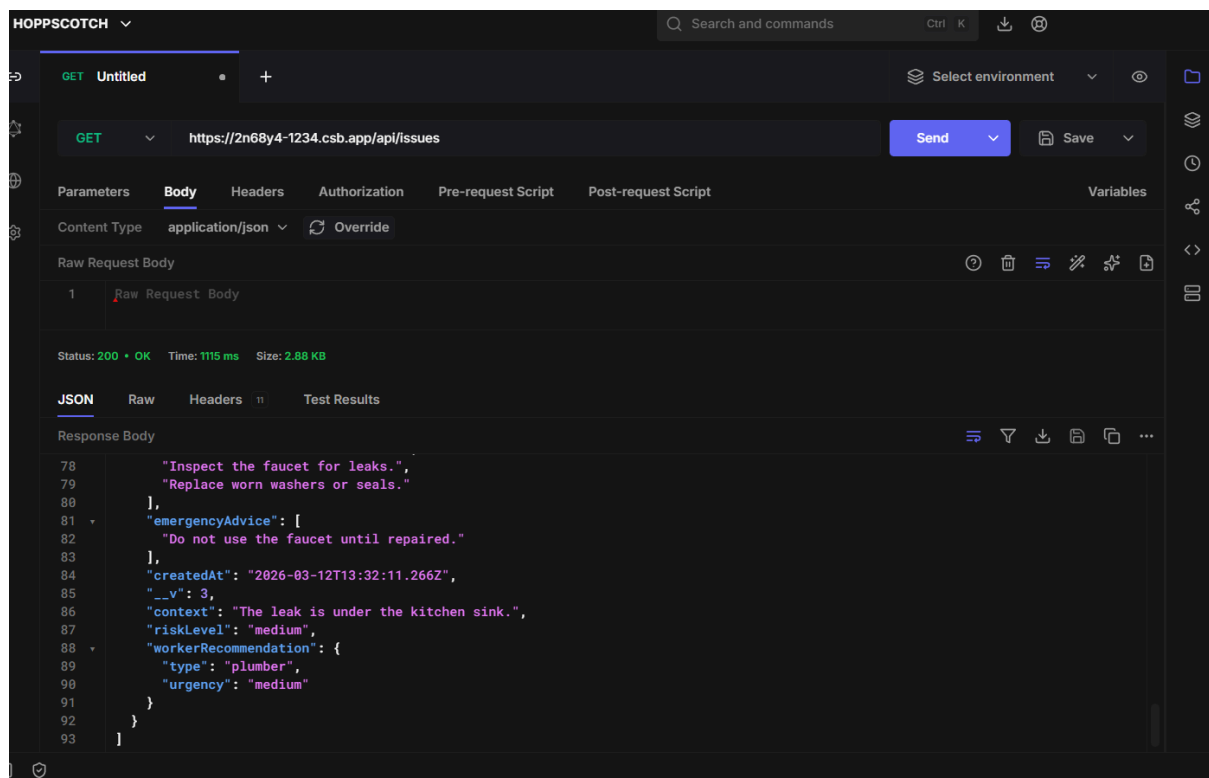
  res.json({
    riskLevel: risk.riskLevel,
    diySteps: solution.diySteps,
    workerRecommendation: solution.workerRecommendation,
    emergencyAdvice: solution.emergencyAdvice,
  });
} catch (err) {
  res.status(500).json({ error: err.message });
}
}

```



- Endpoint URL: `https://2n68y4-1234.csb.app/api/answer`
- HTTP Method: `post`
- Headers: None
- Body:

```
{  "issueId": "69b2bba480220d23574d7bbb",  "answer": "The leak is near the faucet joint."}
```



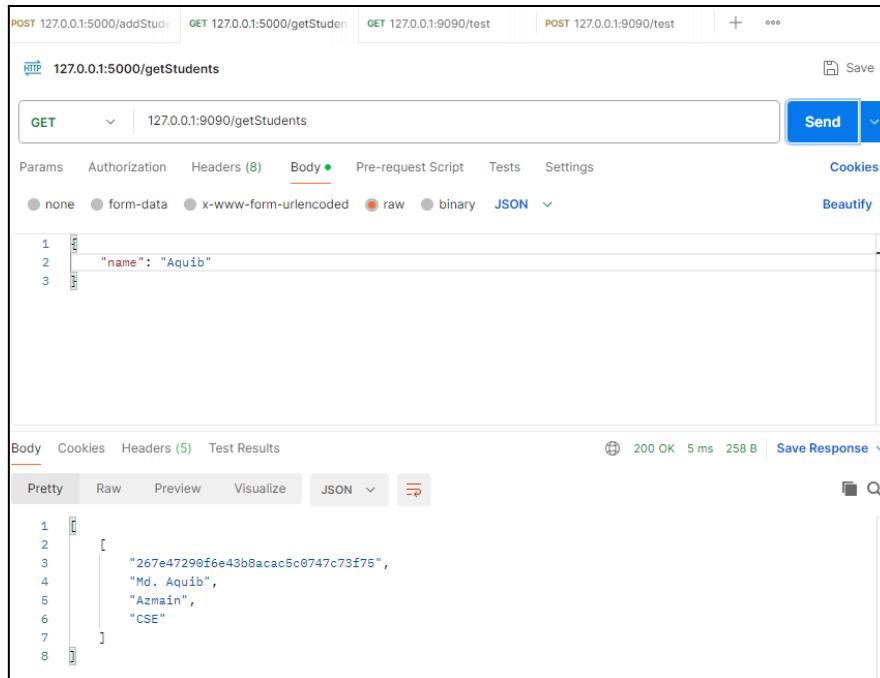
- Endpoint URL: https://2n68y4-1234.csb.app/api/issues
- HTTP Method: Get
- Headers: None
- Body: None

Instruction:

1. You have to develop all the REST APIs/Endpoints of two features (which interfaces you have submitted in the Figma assignment) of your project using your preferred backend framework. **Database must be connected.**
2. Each API description will include:
 - Endpoint URL (e.g., https://api.example.com/users)
 - HTTP Method (GET, POST, PUT, DELETE, etc.)
 - Headers (senlike Content-Type or Authorization)
 - Body/Parameters (data tod with request)
3. This is an individual submission. The developed APIs can not be duplicated among group members.
4. Add a code snippet for each API.
5. Run a backend server on a specific port. The port number should be the **last four digits of your student id.**
6. Test your developed API in Postman. Add a screenshot for each API.

7. For example:

```
@app.route('/getStudents', methods=['GET'])
def getStudents():
    c = sqlite3.connect("student.db").cursor()
    c.execute("SELECT * FROM STUDENTS")
    data = c.fetchall()
    return data
```



- Endpoint URL: 127.0.0.1:9090/getStudents
- HTTP Method: Get
- Headers: None
- Body: {"name": "Aquib"}